Introduction To Django

Prepared By: Ronak Panchal, VTCMCA

Blog: http://drronakpanchal.wordpress.com

django

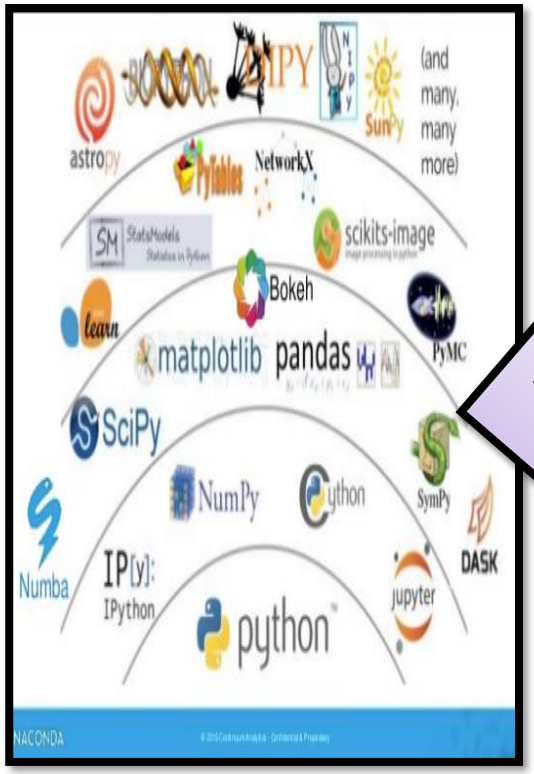
By : Ronak Panchal, VTCMCA

Django is a free and open source **web application framework** which offers fast and effective dynamic website development.

What is framework??????????

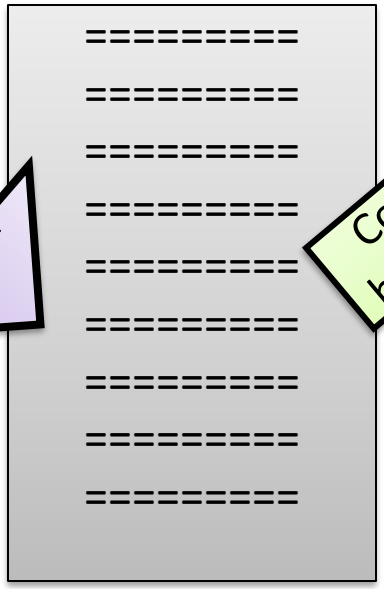
A **web framework** is a code library that makes web development faster and easier by providing common patterns for building reliable, scalable and maintainable web applications

To understand it more clear lets understand the difference between library and framework



Python Library

Library Called by code



Program Code

Code Called by Framework

Framework

Characteristics of Django

- Loosely Coupled – Django helps you to make each element of its stack independent of the others.
- Less code - Ensures effective development
- Not repeated- Everything should be developed in precisely one place instead of repeating it again
- Fast development- Django's offers fast and reliable application development.
- Consistent design - Django maintains a clean design and makes it easy to follow the best web development practices.

Some of the popular sites that uses Django are :

- Pinterest (*tool for collecting and organizing your favorite things*) ,
- Instagram ,
- Knight Foundation,
- Mozilla,
- National Geographic
- Open Knowledge Foundation
- Disqus (*most popular discussion system)*
- *Chess etc*

Django MVT (Model –View-Template)

The MVT is a software design pattern which includes three important components Model, View and Template.

- The **Mode**l helps to handle database. It is a data access layer which handles the data.
- The **Template** is a presentation layer which handles User Interface part completely.
- The **View** is used to execute the business logic and interact with a model to carry data and renders a template.

Although Django follows MVC pattern but maintains it's own conventions. Here control is handled by the framework itself.

Flow of Control in MODEL-VIEW-TEMPLATE

USER

Sends requests for a resorce

Django

Checks availability

URL

VIEW

MODEL

View interacts with Model & Template. Then renders a template

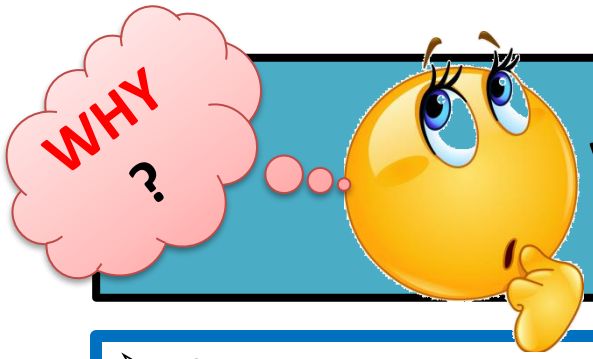
If url is mapped View is called

TEMPLATE

At last Django responds back to the user and sends a template as a response.

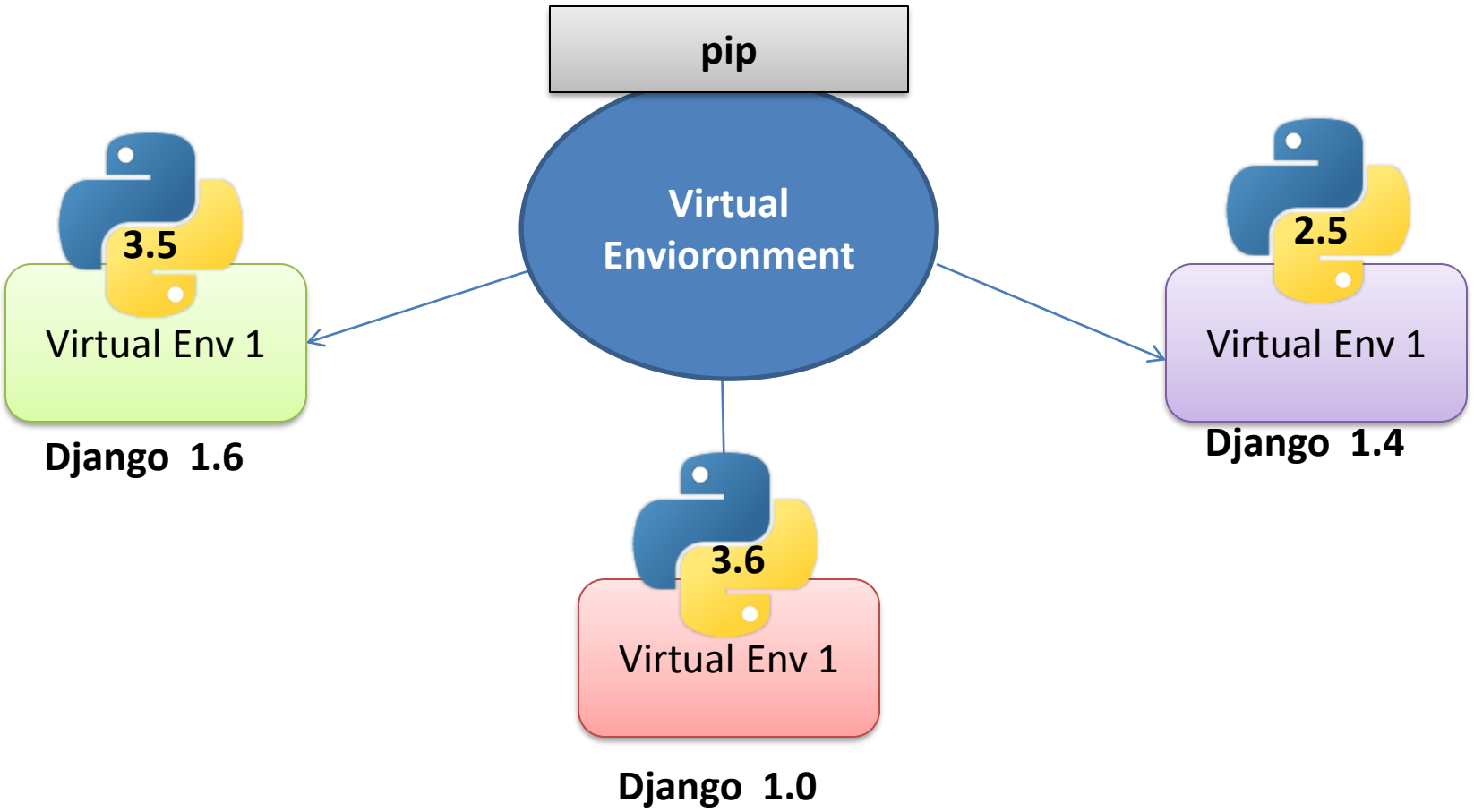
Now our very first step is to create Virtual Envioronment

- The virtual environment is an environment which is used by Django to execute an application. It is recommended to create and execute a Django application in a separate environment.



Virtual Environment

- Its **recommended** to install **Virtualenv** before installing Django .
- Virtual Environment creates isolated environments to isolates your Python files on a per-project basis.
- **It ensure that** any changes **created** to your **web site** won't **have an effect on alternative** websites you're developing.
- The **attention-grabbing half** is **that you just will produce** virtual environments with **completely different** python versions, with every environment having its own set of packages.


pip
Virtual
Envioronment
3.5
Virtual Env 1
Django 1.6
2.5
Virtual Env 1
Django 1.4
3.6
Virtual Env 1
Django 1.0

What should be Used ….

Virtual Enviornment or wrapper???

- **virtualenv** is a tool to create isolated Python environments. The **virtualenv** creates a folder which contains all the necessary executables to use the packages that a Python project would need.
- **virtualenvwrapper** is a set of extensions to **virtualenv** tool. The extensions include wrappers for creating and deleting virtual environments and otherwise managing our development workflow, making it easier to work on more than one project at a time without introducing conflicts in their dependencies.

- Using **virtualenv** without **virtualenvwrapper** is a little bit painful because every time we want to activate a virtual environment, so we have to type long command like this:
  - **myproject/env/activate**
- But with **virtualenvwrapper**, we only need to type:
  - **workon myproject**

With windows, to install virtualenviornmentwrapper……. type

- pip install virtualenvwrapper-win

This is the official website

- https://www.djangoproject.com/start/
- https://docs.djangoproject.com/en/3.1/intro/tutorial01/

Steps to install virtual environment and Django

Step 1 : Go to Window Power Shell (Admin) /command window

Step 2: Move to desired drive (C:, D:, F: etc.) by typing driveName : (example → **F:**) and move to folder by typing cd <foldername> cd DjangoProj in this case

Step 3: type

F:\>pip install virtualenvwrapper –win

(to install virtaulenvwrapper in F: drive)

Steps to install virtual environment and Django

Step 4 : Create virtual environment under the folder DjangoProj by typing

> **mkvirtualenv projenv**

Step 5 : now type

>workon projenv

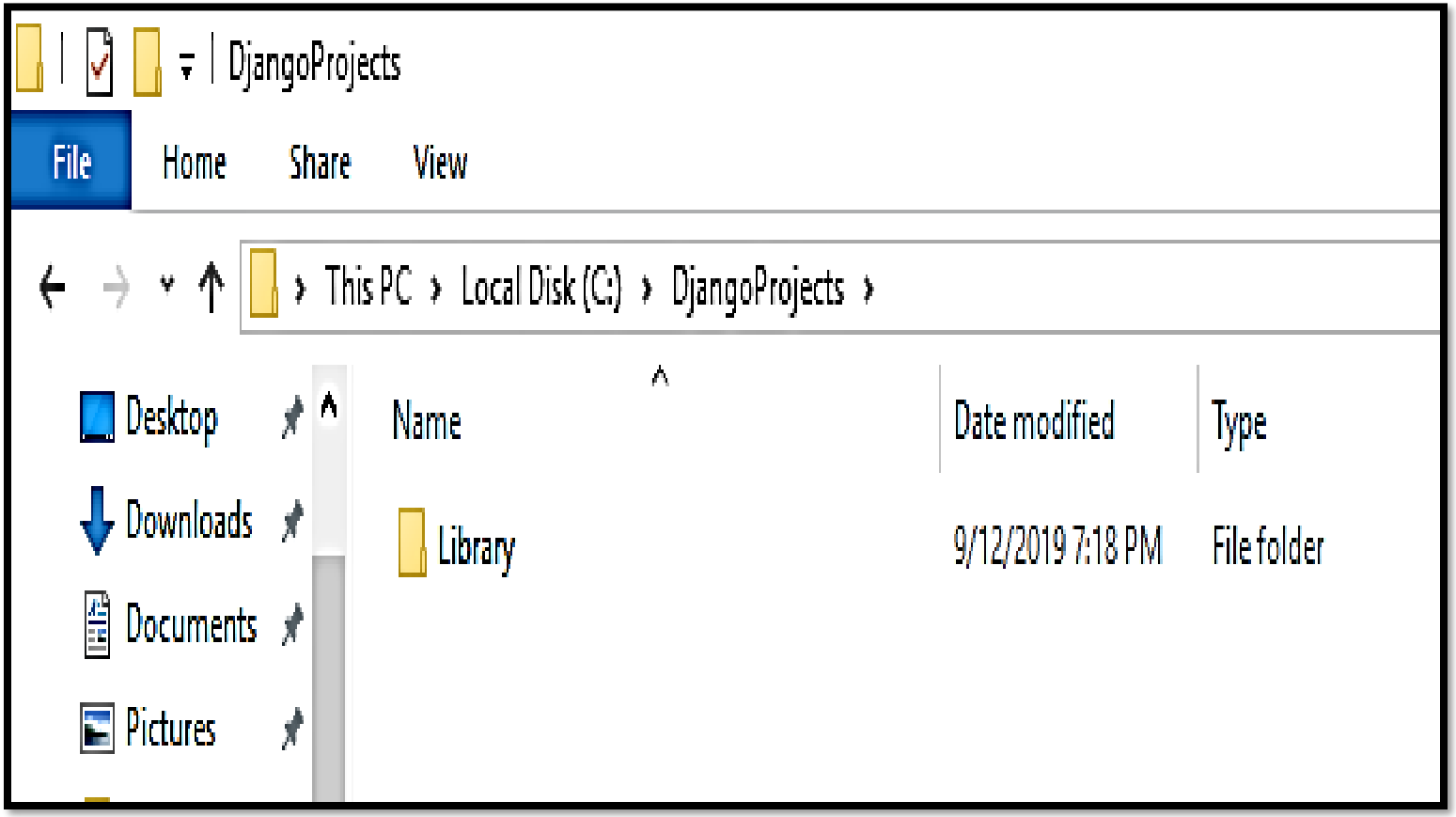
Step 6 : Now install Django by typing

> **pip install django**

Step 7 : Now we will create Django Project namely **LIBRARY**

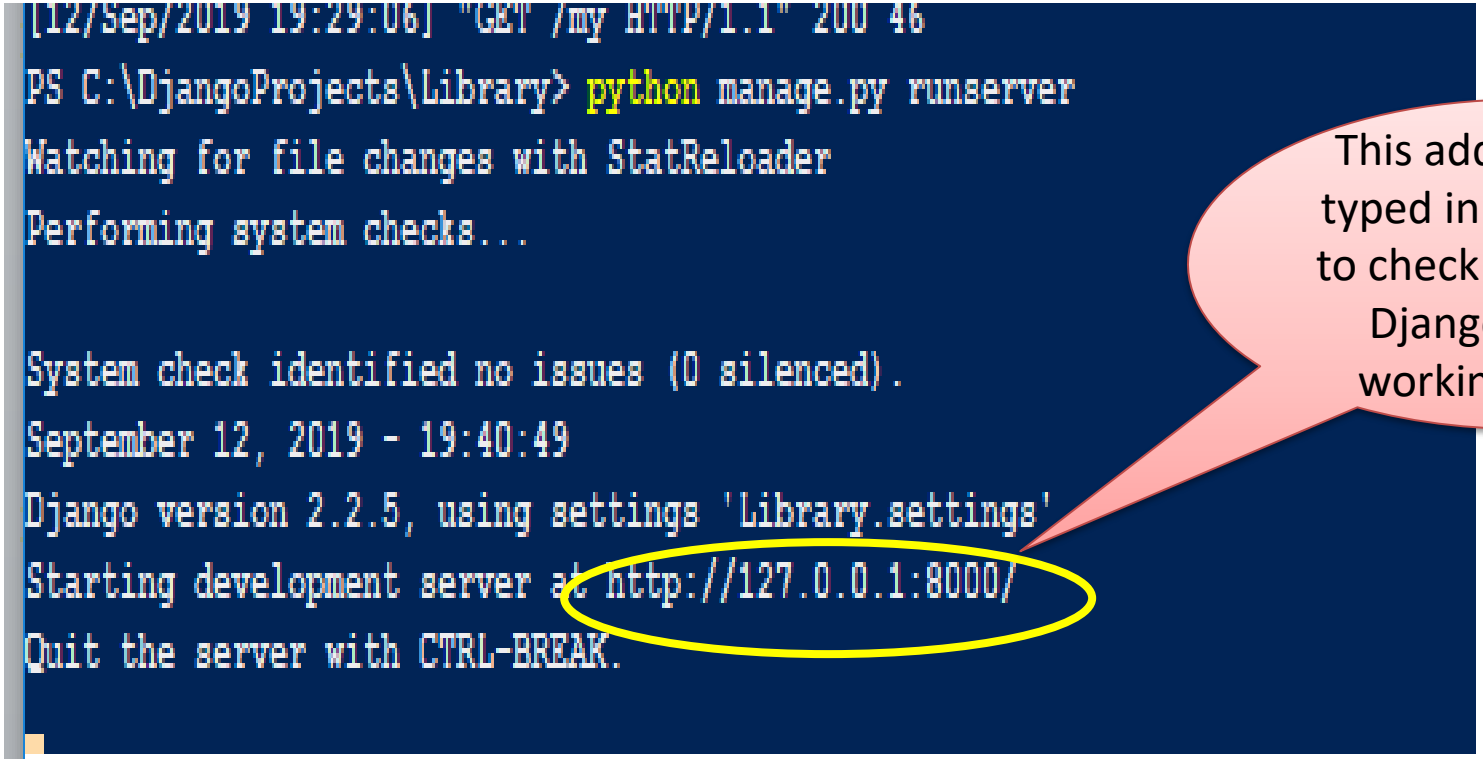
F:\>DjangoProj\MyProj> **django- admin startproject Library**

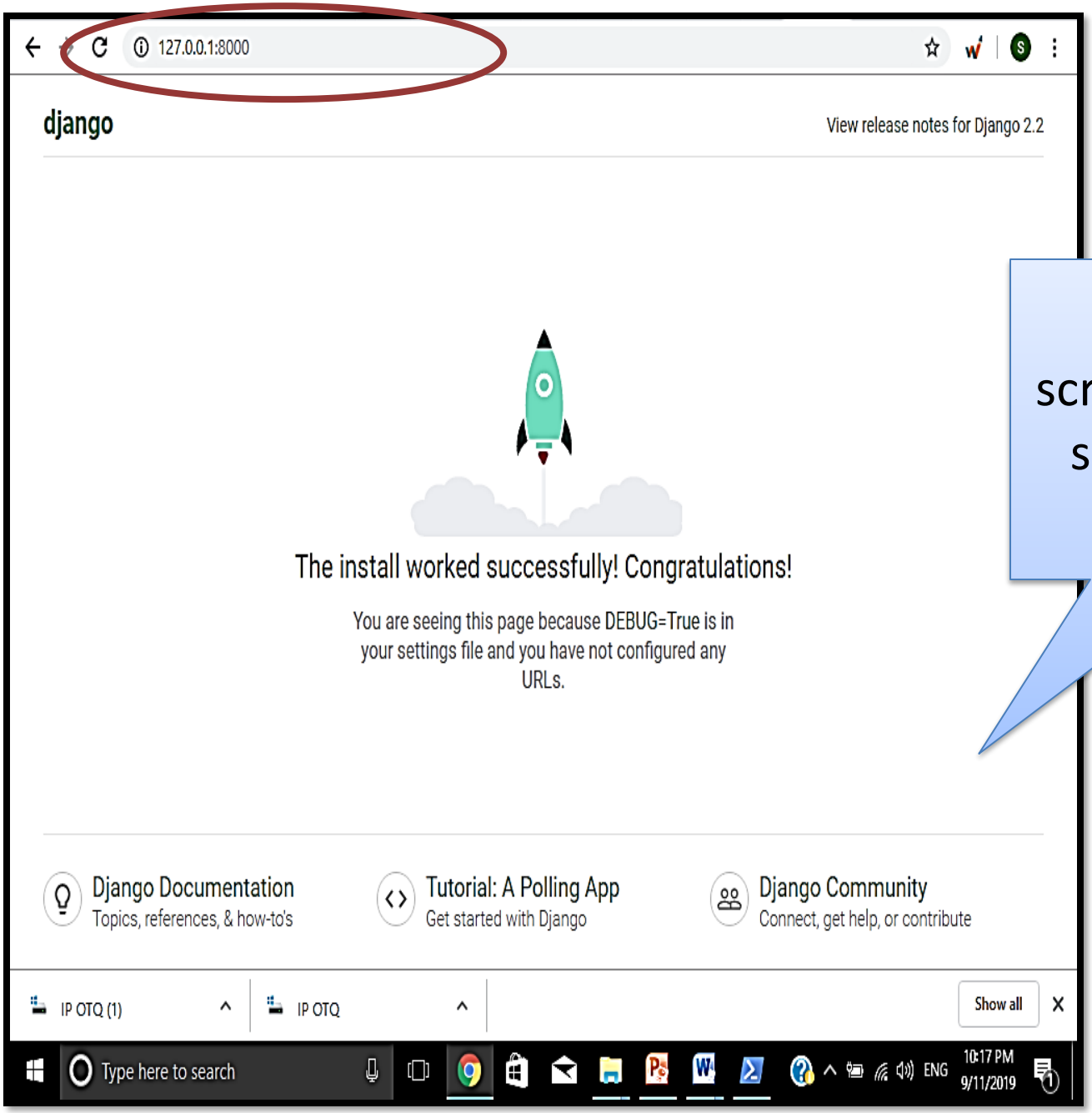
A folder Library will be created under f:\DjangoProj it will look like this :



Now to check whether the Django Server is running or not :

F:\DjangoProj\Library > python manage.py runserver



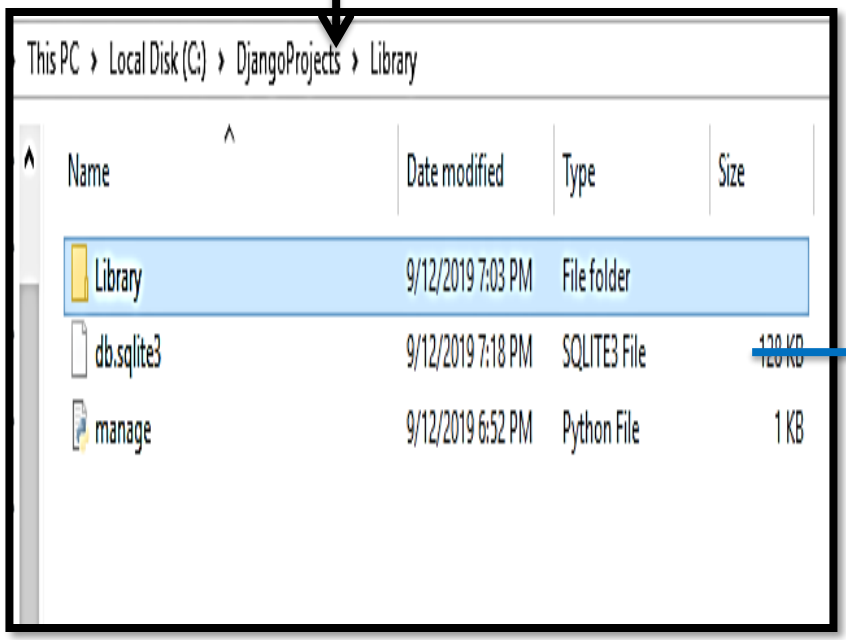


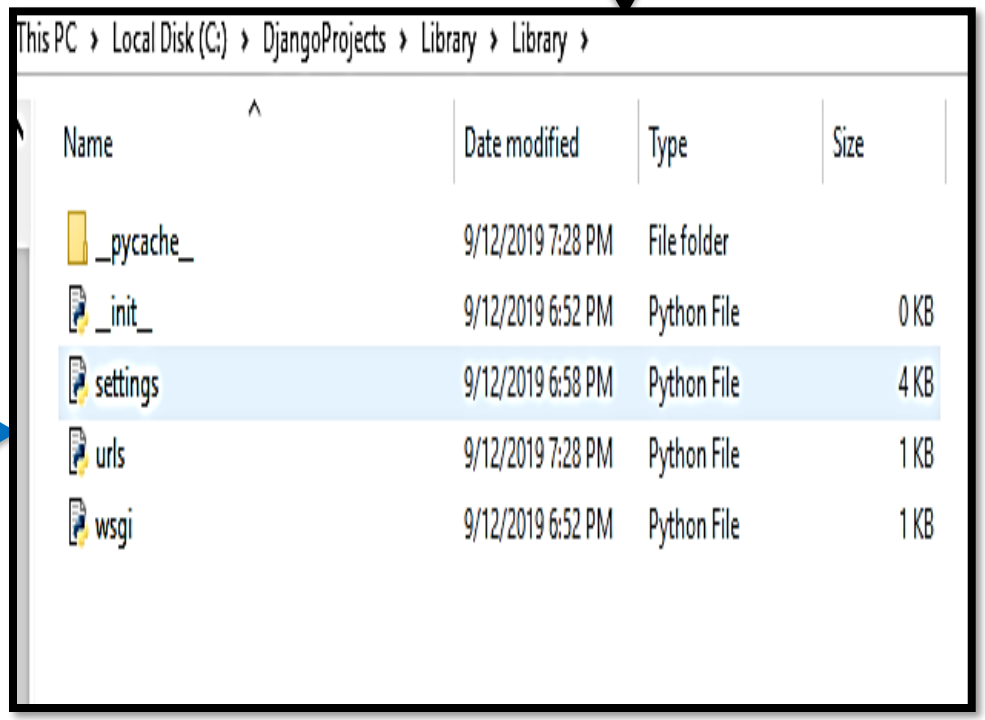
If you see this screen, your Django server is running successfully

Now you will notice that following **Django enviornment** will be created ……

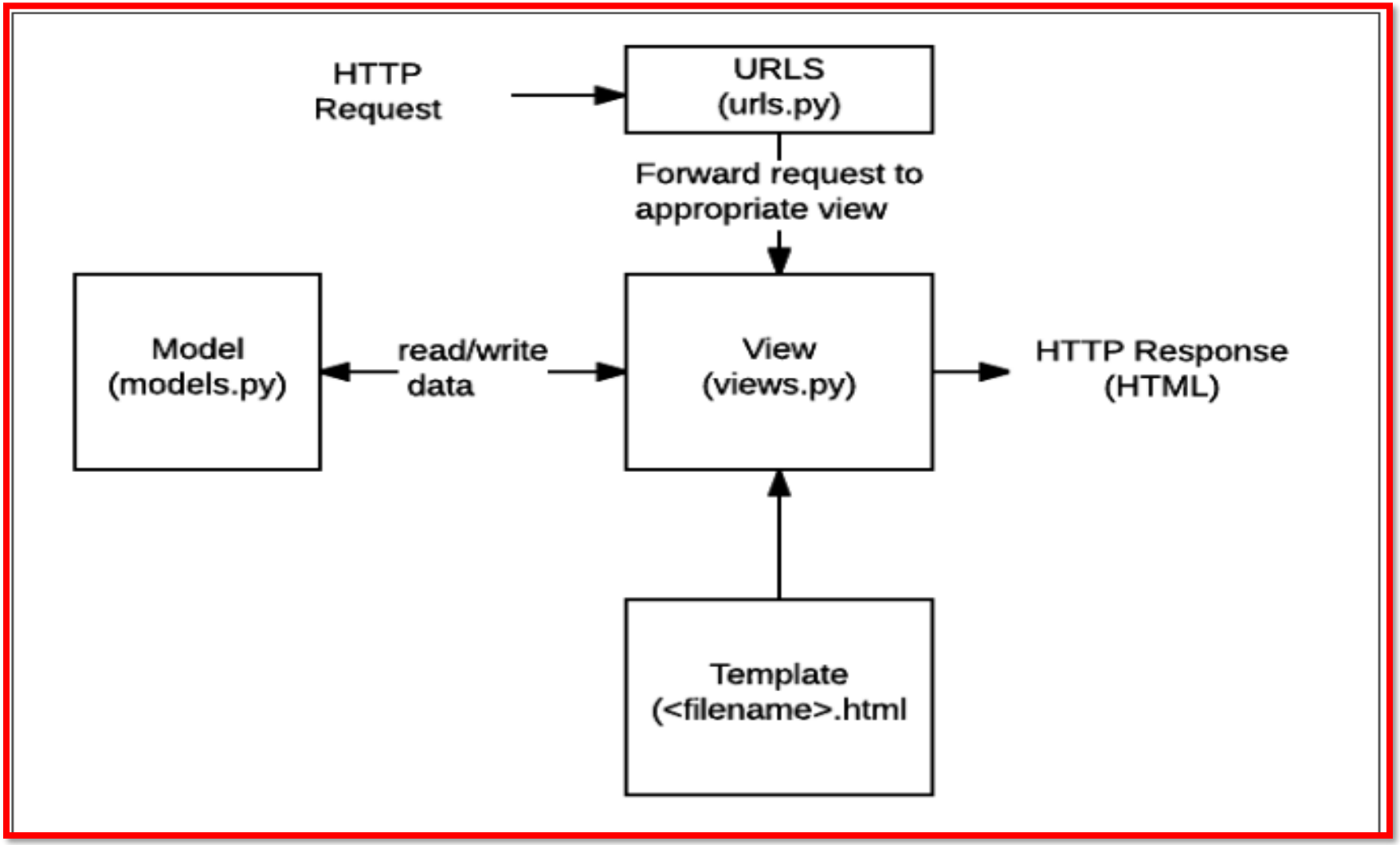
Inside the main Library folder 1 subfolder with same name and 1 file 'manage.py'will be created

Inside the subfolder LIbrary 4 files will
__init__.py
settings.py
urls.py
wsgi.py **will be created**





How these files interact each other



Django Project Structure

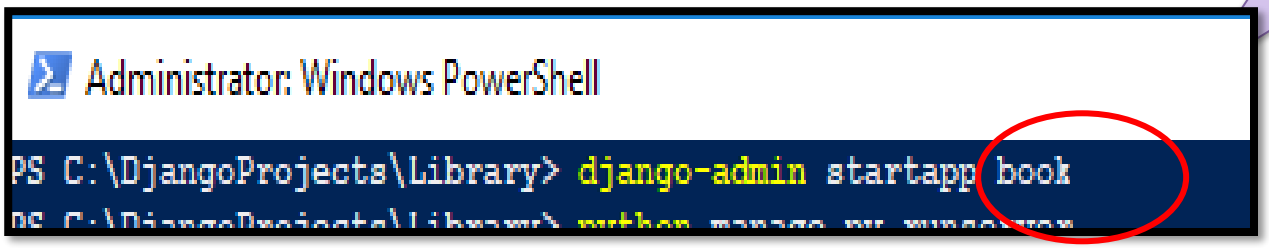
- **manage.py**: a shortcut to use the **django-admin** command-line utility. It's used to run management commands related to our project. We will use it to run the development server, run tests, create migrations and much more.
- **__init__.py**: It is an empty file that indicates that this folder is a Python package.
- **settings.py**: It contains all the project's configuration.
- **urls.py**: With the help of this file we can map the routes and paths in our project. For example, if you want to show something in the URL /about/, you have to map it here first.
- **wsgi.py**: this file is a simple gateway interface used for deployment. We have nothing to do with it.

After creating Project lets create Django App

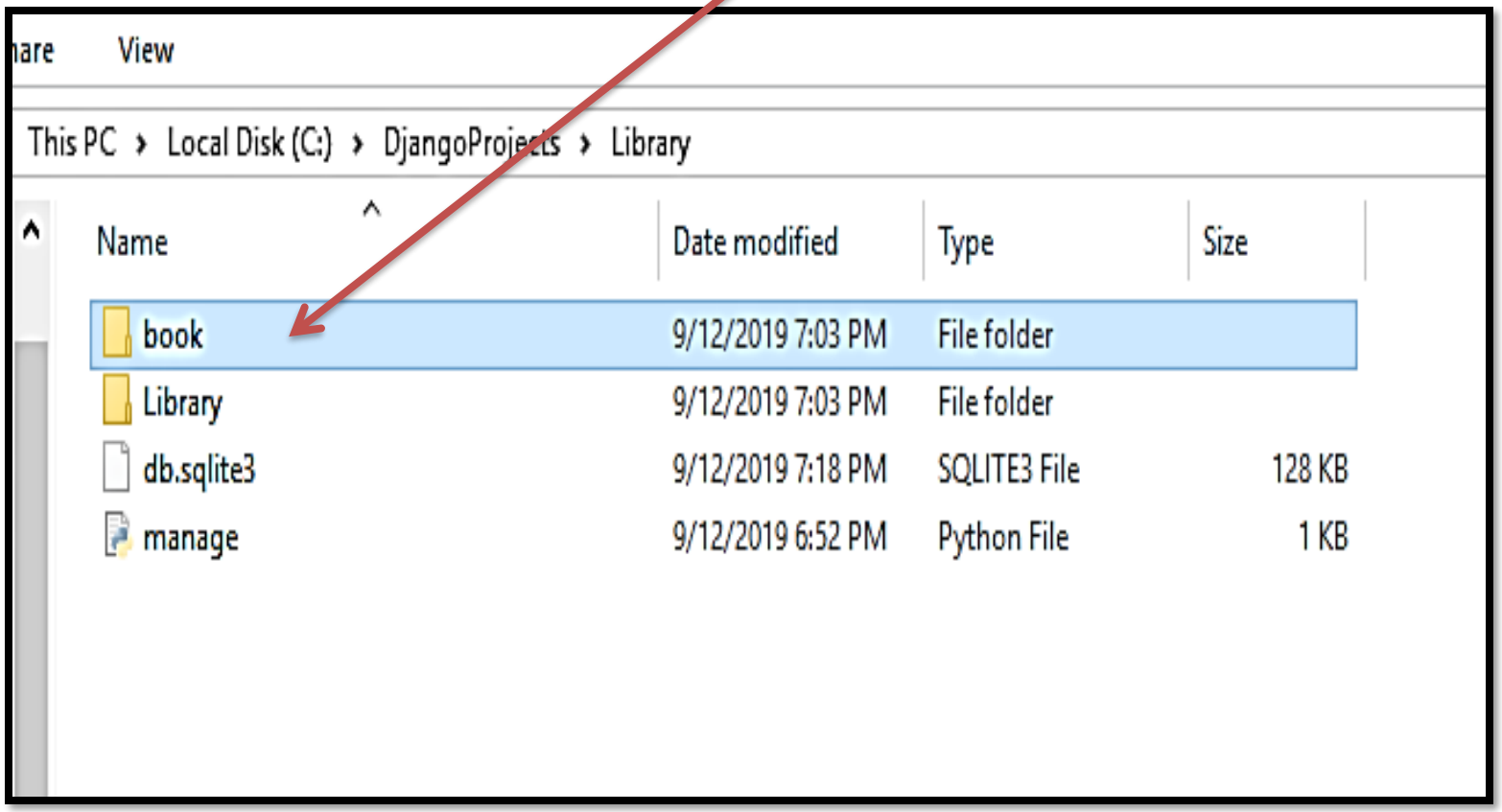
- Firstly make sure your Django server is not running , if it is so stop it by pressing ctrl+C
- Now again switch to Command window or power shell
- Now move to the folder Proj1 (main folder)
- Here type

  > django-admin startapp book

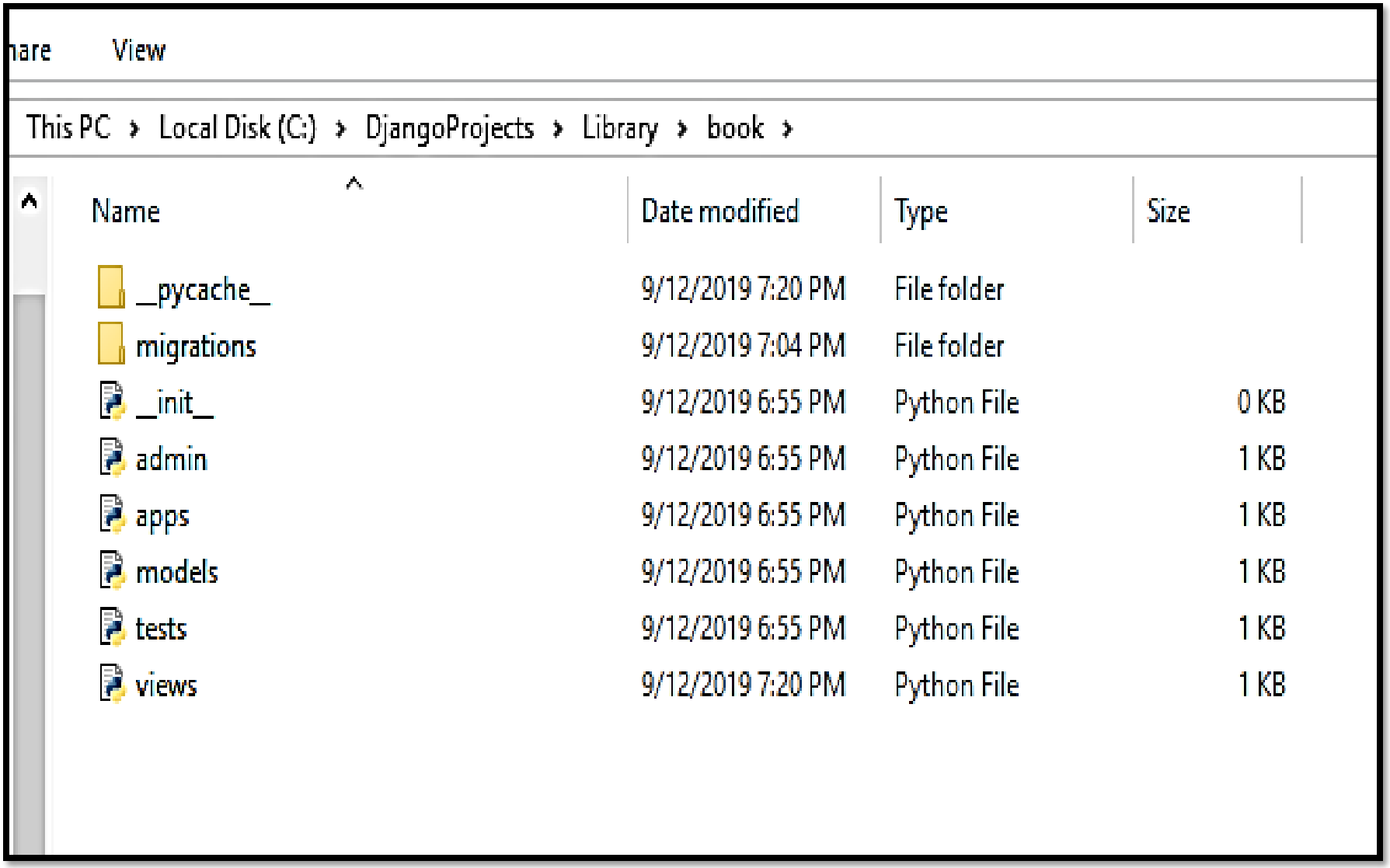
App Name can be given as per your choice



Django App “book” created now



In “book” App following files will be created automatically

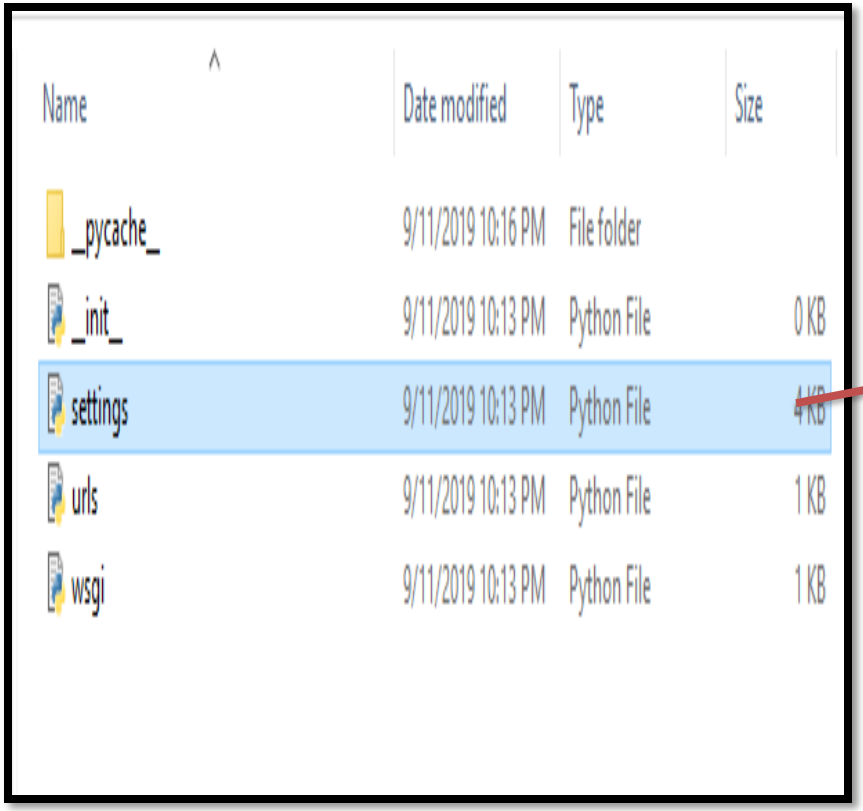


Lets understand working of these files

- **migrations/**: This folder stores some files to keep track of the changes done in **models.py** file, so to keep the database and the **models.py** updated.
- **admin.py**: It is a configuration file for a built-in Django app called **Django Admin**.
- **apps.py**: It is a configuration file of the current app.
- **models.py**This is the file where we define the entities of our Web application. The models are translated automatically by Django into database tables.
- **tests.py**: used to write unit tests for the app.
- **views.py**: this is the file where we handle the request/response for our Web application.

Now that we created our first app book under the Project **Library**, let's configure our project to *use* it.

open the **settings.py** with IDLE and search for the INSTALLED_APPS variable :



INSTALLED_APPS VARIABLE

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Now create any .html file which you want to show with Django and place it under the new folder ‘template’ under the folder ‘Library’

Now Open settings.py with IDLE to register newly created app book in INSTALLED_APPS list and add newly created 'template' folder in TEMPLATES list

settings.py - C:\DjangoProjects\Library\Library\settings.py (3.6.5)

File Edit Format Run Options Window Help

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'book',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'Library.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': ['template'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
```

By including App 'book ' and template folder we are telling the project that template folder includes all pages of app 'book' inside it

Now write code in views file to call html file

views.py - C:\DjangoProjects\Library\book\views.py (3.6.5)

File Edit Format Run Options Window Help

```
from django.shortcuts import render

# Create your views here.

def bookview(request):
        return render(request,'FIRST.html')
```

render() is a special Django helper function that creates a shortcut for communicating with a web browser

Setting views in url :Now open URL.py file and do the following changes to link files

urls.py - C:\DjangoProjects\Library\Library\urls.py (3.6.5)

File Edit Format Run Options Window Help

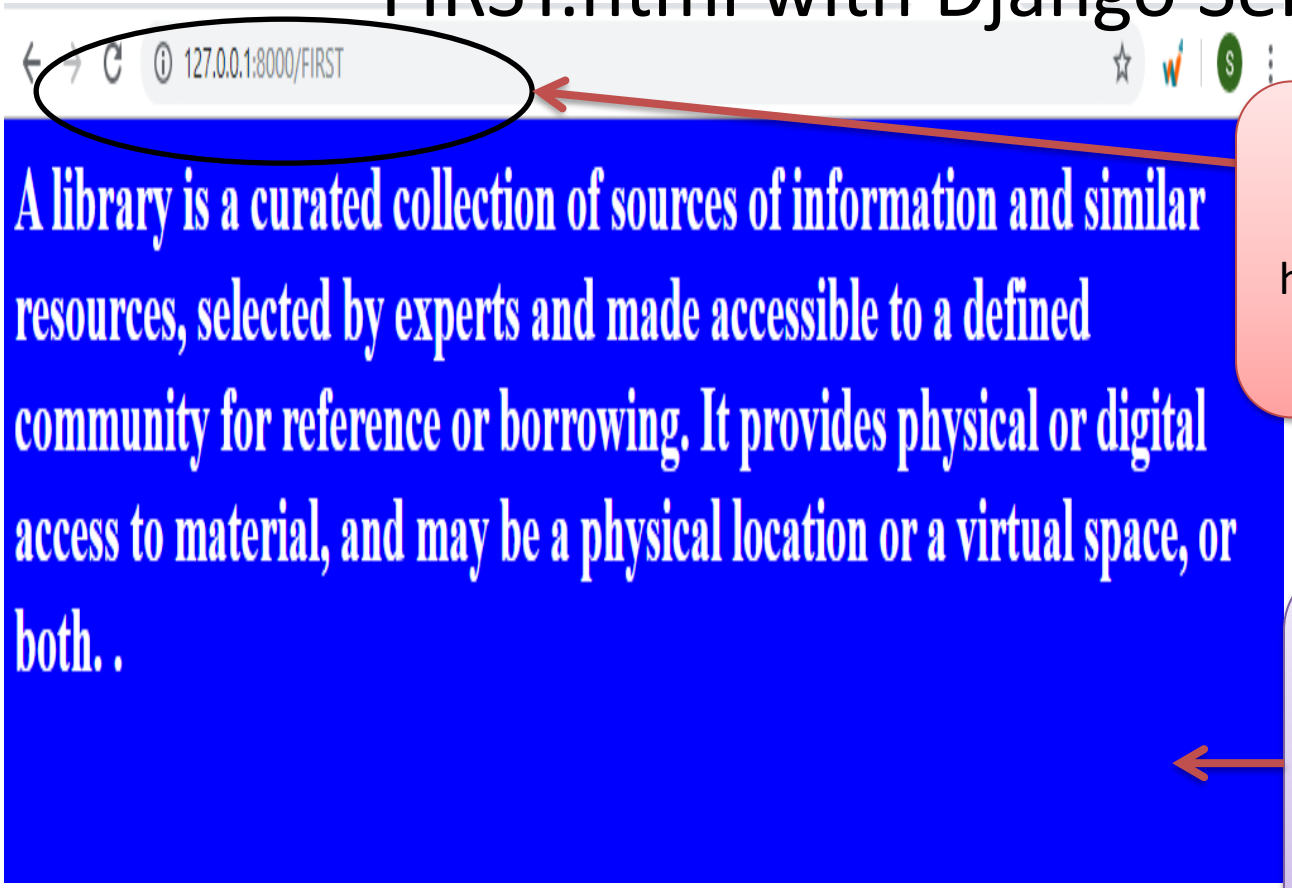
```
"""
from django.contrib import admin
from django.urls import path
from book import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('FIRST',views.bookview),
]
```

Write import statement to include views from book

Here we are including name of webpage (FIRST) to be executed with server and function to be called (bookview)

Now we are ready to run our webpage FIRST.html with Django Server



Type in the web browser http://127.0.0.1:8000/FIRST

Then this web page will be displayed

Hurray !!!!! we have developed our first basic WebPage Successfully.

Lets create a webpage which sends Some Data about books through form and we will save it in database
(GET & POST) Method

Before we start lets understand the basic concepts required to develop such application

What is HTTP?
HTTP is a set of protocols designed to enable communication between clients and servers. It works as a request-response protocol between a client and server. A web browser may be the client, and an application on a computer that hosts a web site may be the server.

To request a response from the server, there are mainly two methods:

- **GET** : to request data from the server.
- **POST** : to submit data to be processed to the server.

The post() method is used when you want to send some data to the server.

- Syntax
- requests.post(*url*, data={*key*: *value*}, json={*key*: *value*}, *args*)

Lets Start to ……

We are going to continue with previously created project “Library” and App “Book”

Now first step is to create HTML form to input book details..

Here we have created a html file under template folder to create 3 text boxes to input book code, book name and author name….you can add more controls to read more data as per your need

bookdetail - Notepad

File Edit Format View Help

File Name : **bookdetail.html**

```
<HTML>
<BODY>
<FORM action="#" method = "post">
{%csrf_token%}
 <font color="red" ><b> Book Code  : <b><br> <input type="text" name="bcode" > <br>
<br> </font>
<font color="red" >Book Name : <br> <input type="text" name="bnm" > <br>
<br></font>
<font color="red" > Author        : <br> <input type="text" name="auth" > <br>
<br></font>
<input type="submit" value="   SUBMIT  ">

</FORM>
</BODY>
</HTML>
```

Don't forget to add

1. action ="#" (to tellthis form will be processed within file/url)
2. {%csrf token%} in html file

Csrf token is used to send requests to the server, in which the **token** validates them.

Now create a view :

open view.py under the App book and add the following function

views.py - C:\DjangoProjects\Library\book\views.py (3.6.5)

File Edit Format Run Options Window Help

```
from django.shortcuts import render
import csv

# Create your views here.



def bookentry(request):
      if request.method=='POST':
            book_dict=request.POST
            with open ('Books.csv','a') as bk:
                  W=csv.writer(bk)
                  for key,value in book_dict.items():
                        W.writerow([key,value])
      return render(request,'bookdetail.html')
```

Here Bookentry function is created which

1. **store the POSTed data into book_dict dictionary object**
2. **creating a csv file books.csv and writing the data into it**

Setting.py : Here we have already done the required changes with previosly created webpage. So now there is no need to do any update

- Url.py :

urls.py - C:\DjangoProjects\Library\Library\urls.py (3.6.5)

File Edit Format Run Options Window Help

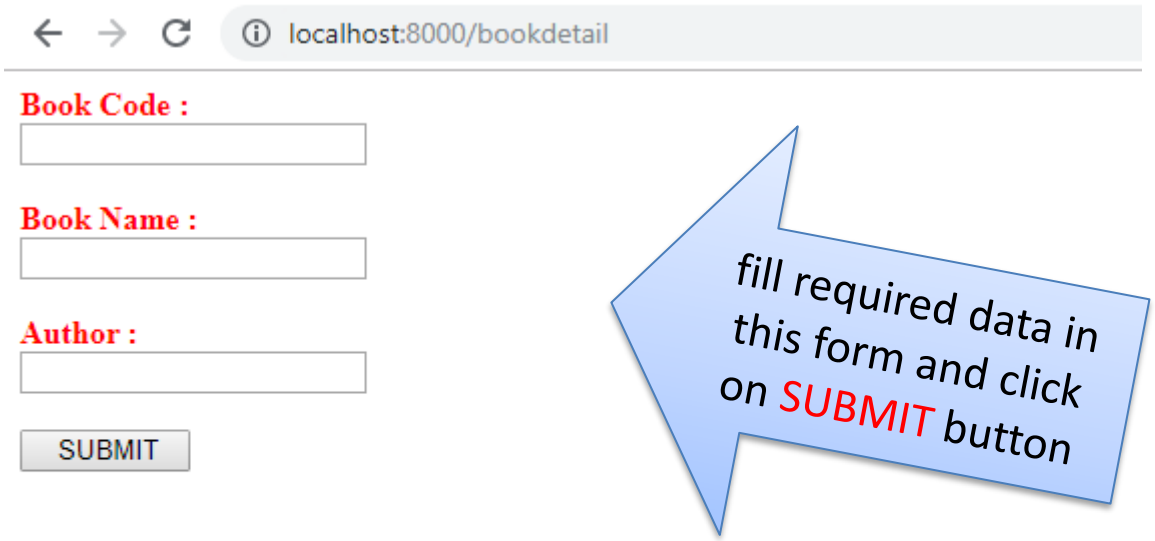
```
"""
from django.contrib import admin
from django.urls import path
from book import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('FIRST',views.bookview),
    path('bookdetail',views.bookentry),
]
```

Add this line

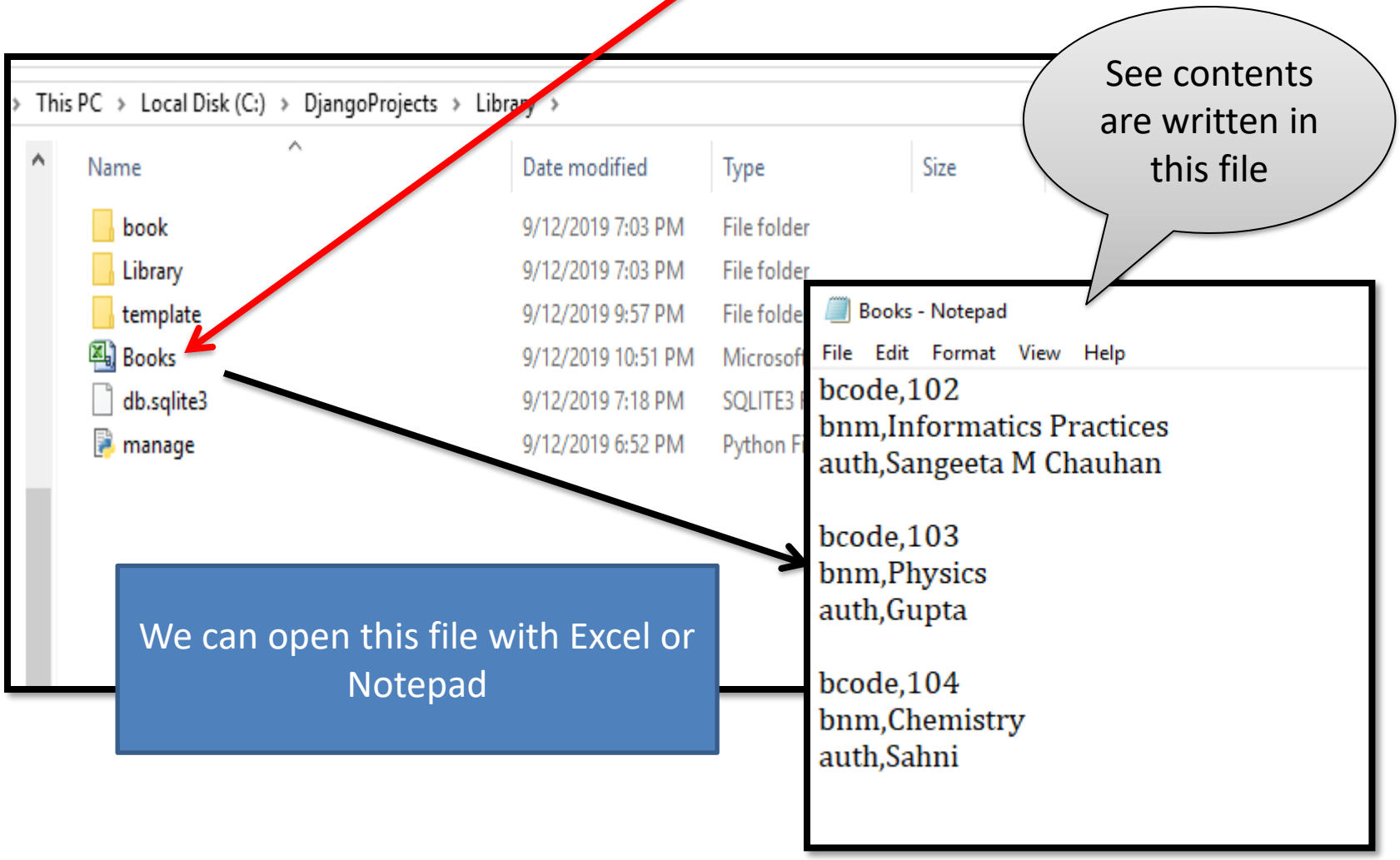
Now we are ready to run this application …Open browser and type localhost:8000/bookdetail in address bar

It will show you form you have created using html “bookdetail.html”



Note : You can add as many record as you want using this form

Now you will notice a new file is created under your project



Now its your turn to create similar application

References

- www.pythonclassroomdiary.wordpress.com